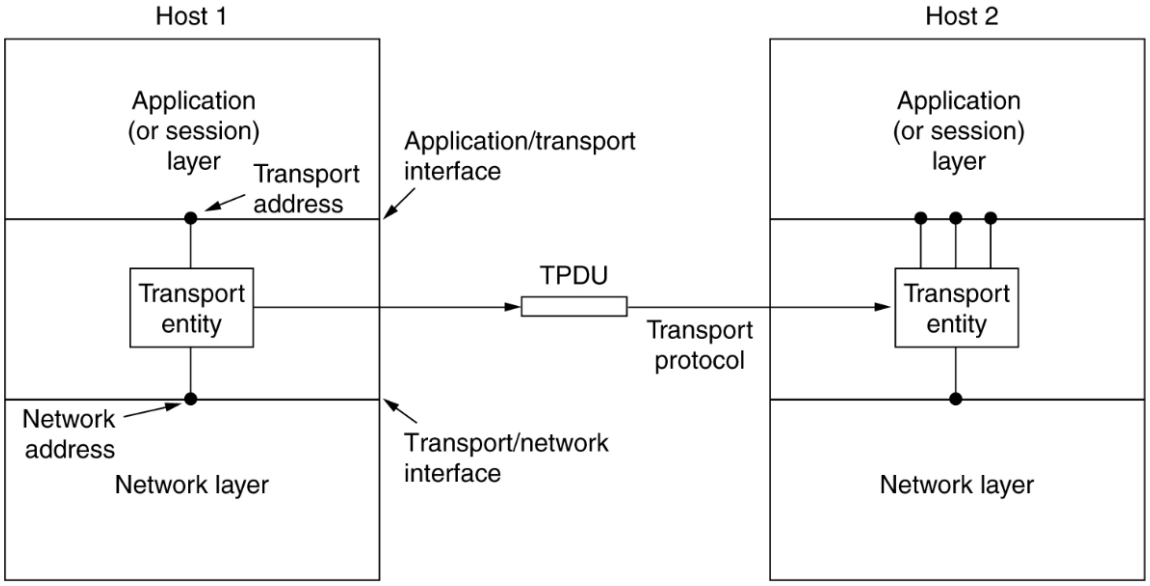
TRANSPORT LAYER

The transport layer builds on the network layer to provide data transport from a process on a source machine to a process on a destination machine with a desired level of reliability that is independent of the physical networks currently in use.

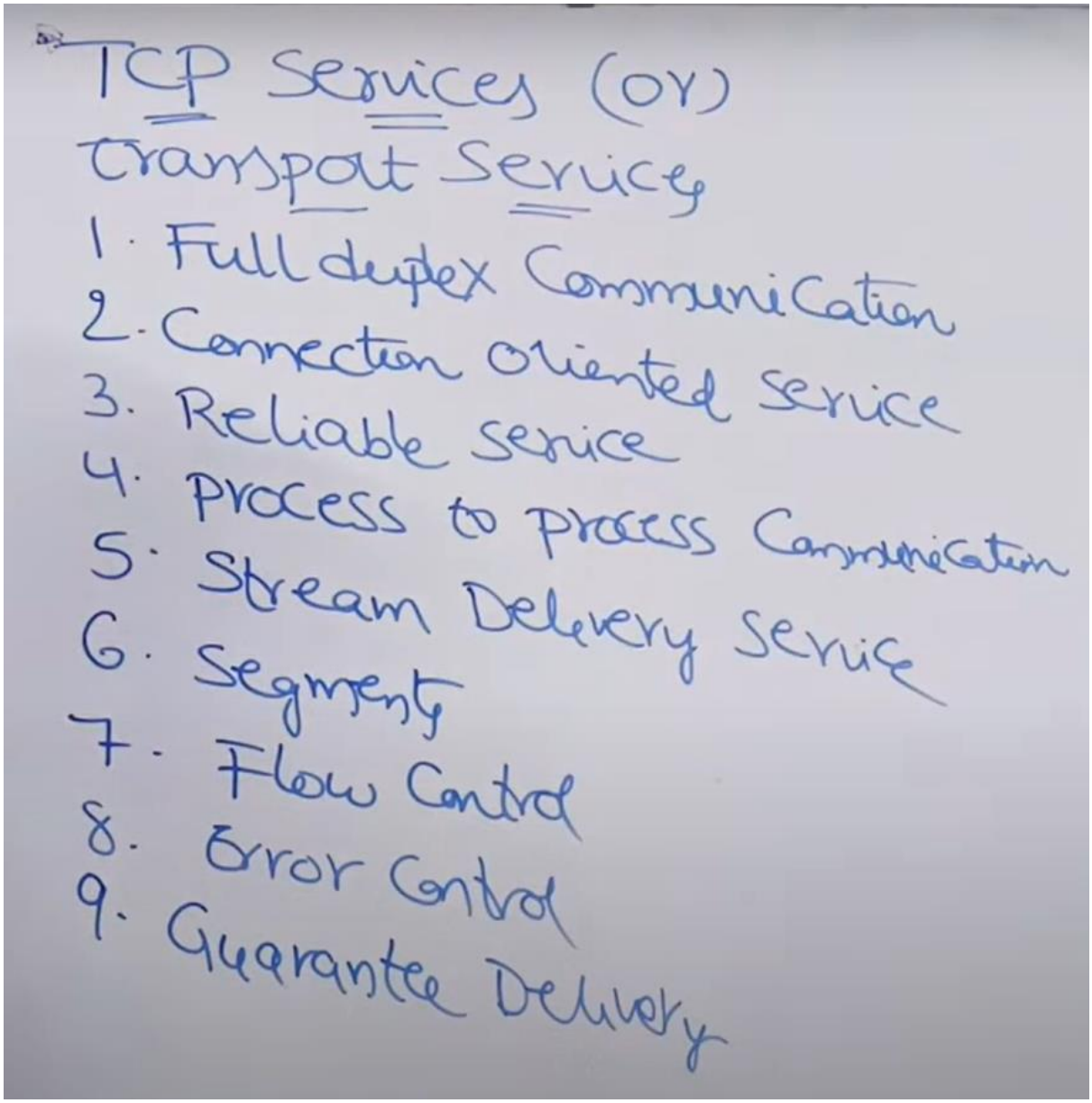
The transport Layer is the second layer in the TCP/IP model and the fourth layer in the OSI model. It is an end-to-end layer used to deliver messages to a host. It is termed an end-to-end layer because it provides a point-to-point connection rather than hop-to-hop, between the source host and destination host to deliver the services reliably. The unit of data encapsulation in the Transport Layer is a segment.

Services Provided to the Upper Layers:

The ultimate goal of the transport layer is to provide efficient, reliable, and cost-effective data transmission service to its users, normally processes in the application layer. To achieve this, the transport layer makes use of the services provided by the network layer. The software and/or hardware within the transport layer that does the work is called the transport entity.



The network, transport, and application layers.



1.Full-Duplex Communication in the transport layer refers to the ability to send and receive data simultaneously between two devices or endpoints. It allows for two-way communication, where both ends of the communication channel can transmit receive data at the same time without interrupting each other.

2.Connection-oriented transport service is one where a reliable connection is established between two endpoints before data transfer can occur. The connection undergoes three main phases: **Establishment**, **Data Transfer**, and **Release**.

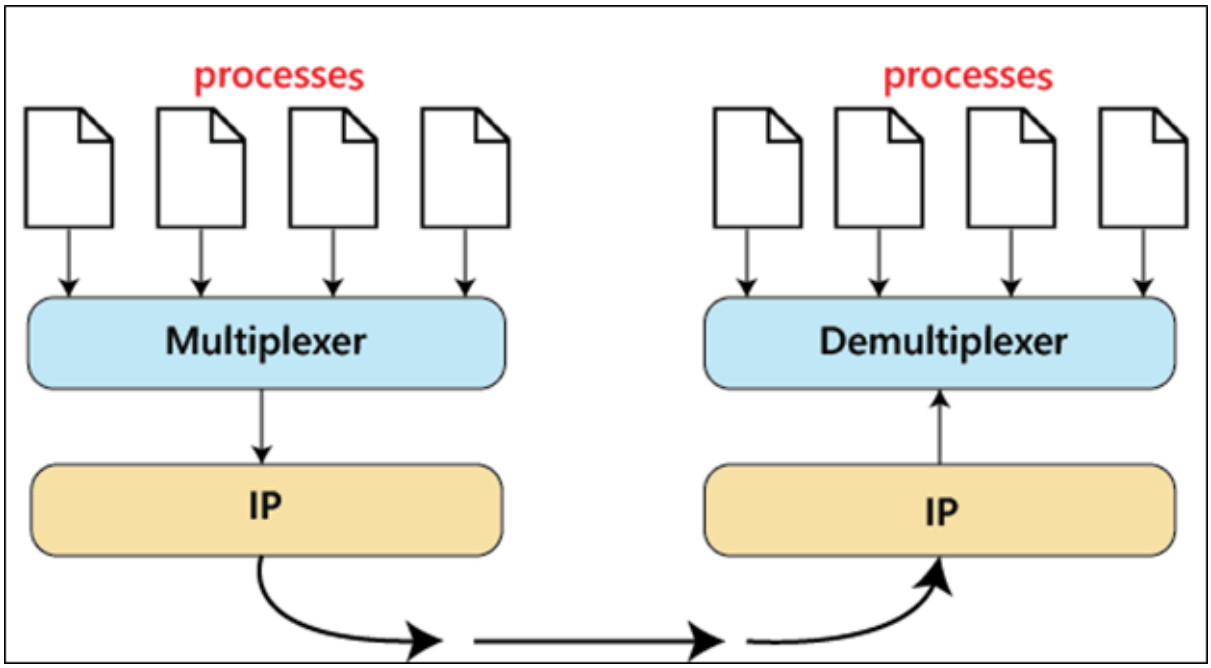
3.Reliable service: The transport layer ensures the reliable transmission of data between systems.

At the sender's side: The transport layer receives data (message) from the Application layer and then performs Segmentation, divides the actual message into segments, adds the source and destination's port numbers into the header of the segment, and transfers the message to the Network layer.

At the receiver's side: The transport layer receives data from the Network layer, reassembles the segmented data, reads its header, identifies the port number, and forwards the message to the appropriate port in the Application layer.

4.Process to Process communication delivery

While Data Link Layer requires the MAC address (48 bits address contained inside the Network Interface Card of every host machine) of source-destination hosts to correctly deliver a frame and the Network layer requires the IP address for appropriate routing of packets, in a similar way Transport Layer requires a Port number to correctly deliver the segments of data to the correct process amongst the multiple processes running on a particular host. A port number is a 16-bit address used to identify any client-server program uniquely.



5.**Stream delivery service** is a type of communication service that continuously delivers a sequence of data, typically in a steady, ordered flow, rather than in discrete chunks. In the context of the transport layer, stream delivery services focus on transmitting data as a continuous stream of bytes between two endpoints, ensuring that the sequence of data remains **unchanged** from sender to receiver.

6. **Segmentation**: Larger data from the application layer is divided into smaller segments at the transport layer, especially in protocols like TCP. This is done because sending data in smaller pieces is more efficient and manageable across networks.

7.Flow control is the process of managing the rate of data transmission between two devices to ensure that the sender does not overwhelm the receiver with more data than it can handle at any given time.

Flow Control in Action:

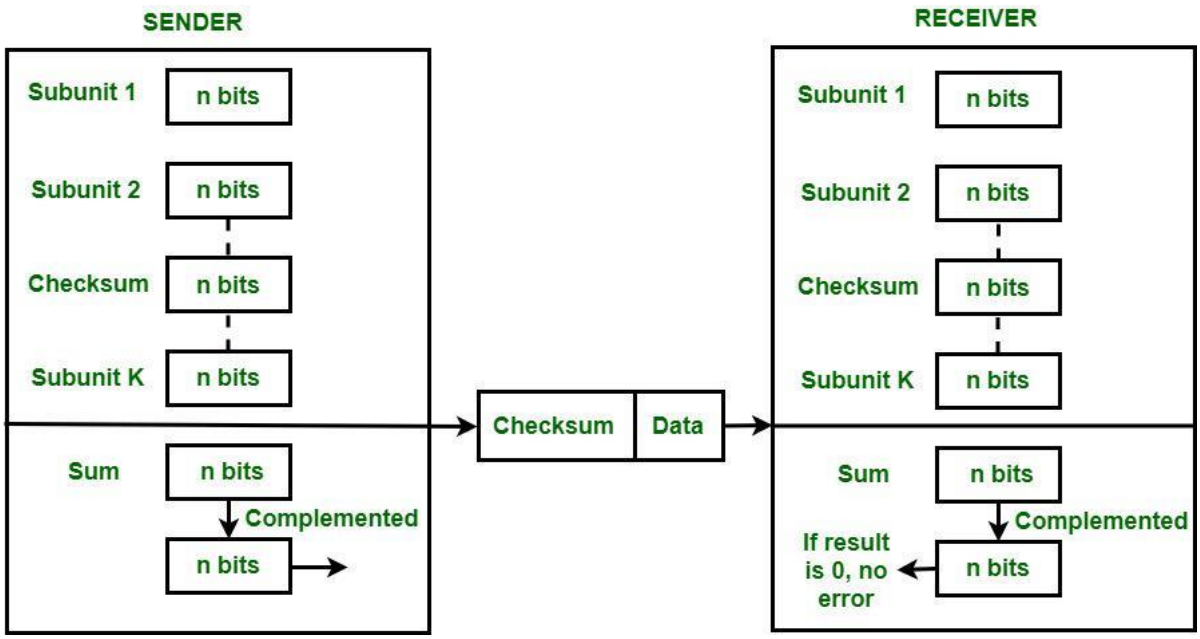
- When the receiver has enough space in its buffer, it sends a larger window size in the acknowledgment to allow the sender to transmit more data.
- If the buffer fills up, the receiver sends a smaller window size, which forces the sender to pause or slow down the data transmission.

- The transport layer provides a flow control mechanism between the adjacent layers of the TCP/IP model. TCP also prevents data loss due to a fast sender and slow receiver by imposing some flow control techniques. It uses the method of sliding window protocol which is accomplished by the receiver by sending a window back to the sender informing the size of data it can receive.

8.Error control ensures that data is transmitted accurately and without corruption. It involves detecting errors during transmission and ensuring that erroneous or lost data is retransmitted.

Error Control in Action:

- The sender sends data with a checksum, and the receiver checks for errors using the same checksum.
- If an error is found, the receiver requests a retransmission, and the sender sends the correct data again.



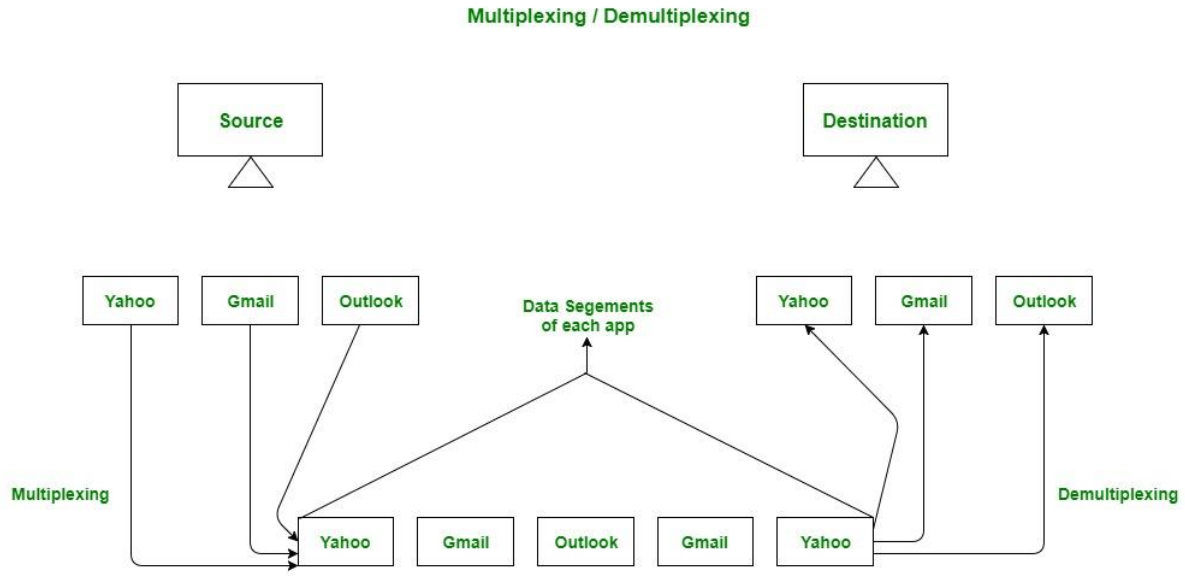
The transport layer checks for errors in the messages coming from the application layer by using error detection codes, and computing checksums, it checks whether the received data is not corrupted and uses the ACK and NACK services to inform the sender if the data has arrived or not and checks for the integrity of data.

9. **Guaranteed Delivery** ensures that data sent from the sender will be delivered to the receiver correctly, without loss or corruption. Protocols like **TCP** achieve this through acknowledgment, retransmission of lost data, error detection, and mechanisms for maintaining the correct order of data.

10.**Multiplexing and Demultiplexing**

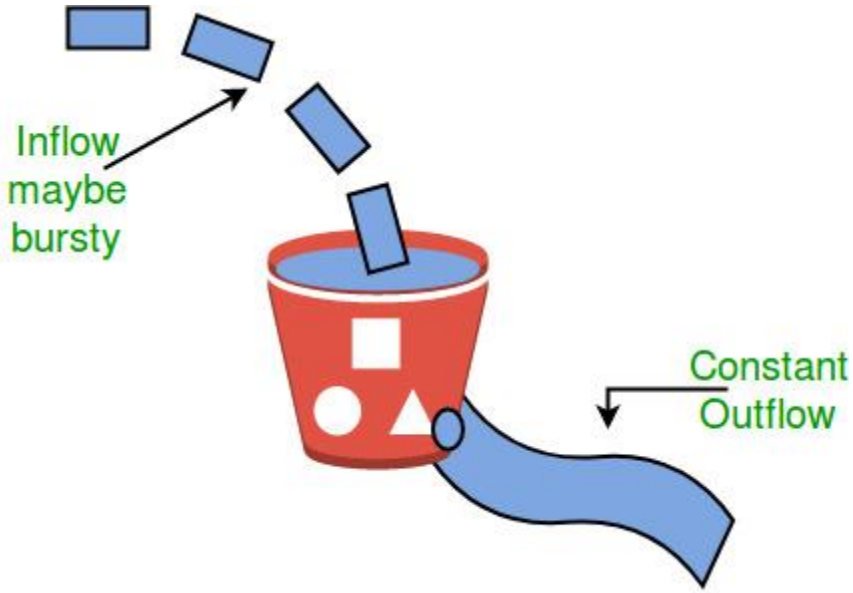
Multiplexing(many to one) is when data is acquired from several processes from the sender and merged into one packet along with headers and sent as a single packet. Multiplexing allows the

simultaneous use of different processes over a network that is running on a host. The processes are differentiated by their port numbers. Similarly, Demultiplexing(one to many) is required at the receiver side when the message is distributed into different processes. Transport receives the segments of data from the network layer distributes and delivers it to the appropriate process running on the receiver's machine.



11.Congestion Control

Congestion is a situation in which too many sources over a network attempt to send data and the router buffers start overflowing due to which loss of packets occurs. As a result, the retransmission of packets from the sources increases the congestion further. In this situation, the Transport layer provides Congestion Control in different ways. It uses open-loop congestion control to prevent congestion and closed-loop congestion control to remove the congestion in a network once it occurred..



Transport Service Primitives:

Transport Service Primitives

Primitive	Packet sent	Meaning
LISTEN	(none)	Block until some process tries to connect
CONNECT	CONNECTION REQ.	Actively attempt to establish a connection
SEND	DATA	Send information
RECEIVE	(none)	Block until a DATA packet arrives
DISCONNECT	DISCONNECTION REQ.	This side wants to release the connection

To start with, the server executes a LISTEN primitive, typically by calling a library procedure that makes a system call that blocks the server until a client turns up. When a client wants to talk to the server, it executes a CONNECT primitive. The transport entity carries out this primitive by blocking the caller and sending a packet to the server. The client's CONNECT call causes a CONNECTION REQUEST segment to be sent to the server. When it arrives, the transport entity checks to see that the server is blocked on a LISTEN (i.e., is interested in handling requests). If so, it then unblocks the server and sends a CONNECTION ACCEPTED segment back to the client. When this segment arrives, the client is unblocked and the connection is established. Data can now be exchanged using the SEND and RECEIVE primitives. In the simplest form, either party can do a (blocking) RECEIVE to wait for the other party to do a SEND. When the segment arrives, the receiver is unblocked. It can then process the segment and send a reply. As long as both sides can keep track of whose turn it is to send, this scheme works fine.

When a connection is no longer needed, it must be released to free up table space within the two transport entities. Disconnection has two variants: asymmetric and symmetric. In the asymmetric variant, either transport user can issue a DISCONNECT primitive, which results in a DISCONNECT segment being sent to the remote transport entity. Upon its arrival, the connection is released. In the symmetric variant, each direction is closed separately, independently of the other one. When one side does a DISCONNECT, that means it has no more data to send but it is still willing to accept data from its partner. In this model, a connection is released when both sides have done a DISCONNECT.

Berkeley Sockets

Berkeley Sockets (often just called **sockets**) are a fundamental programming interface used for network communication. They provide a set of functions that allow a program to communicate with other programs over a network, using either **TCP/IP** or **UDP** protocols. Berkeley Sockets originated from the **Berkeley Software Distribution (BSD)** Unix operating system and have since become a standard API for network communication across various platforms.

Key Concepts of Berkeley Sockets:

1. **Socket**:

- A **socket** is an endpoint for communication between two machines over a network.
- Sockets are used for inter-process communication (IPC) and can be thought of as a combination of an **IP address** and a **port number**. A **socket address** is a combination of an IP address and a port number. It uniquely identifies a destination on the network Where as IP addresses identify devices on the network, and port numbers identify the specific application or service running on those devices.
- A socket typically represents one side of a communication channel. The pair of sockets on both ends of a network connection establishes a full-duplex communication channel.

2.Primary types of Berkeley Sockets:

SOCK_STREAM (Stream Sockets):

- Protocol: Typically used with TCP (Transmission Control Protocol).
- Nature: Connection-oriented and reliable communication.
- Data Delivery: Guarantees ordered, error-free, and reliable delivery of data.
- Use Case: Used in applications where reliable communication is required, such as web browsers, email clients, FTP, and other client-server applications.
- Example: HTTP (HyperText Transfer Protocol) for web browsing.

SOCK_DGRAM (Datagram Sockets):

- Protocol: Typically used with UDP (User Datagram Protocol).
- Nature: Connectionless and unreliable communication.
- Data Delivery: Does not guarantee order or reliability of data delivery.
- Use Case: Used in applications where speed is more important than reliability, such as video streaming, online gaming, DNS (Domain Name System), or real-time communications.
- Example: Online multiplayer games, live broadcasts.

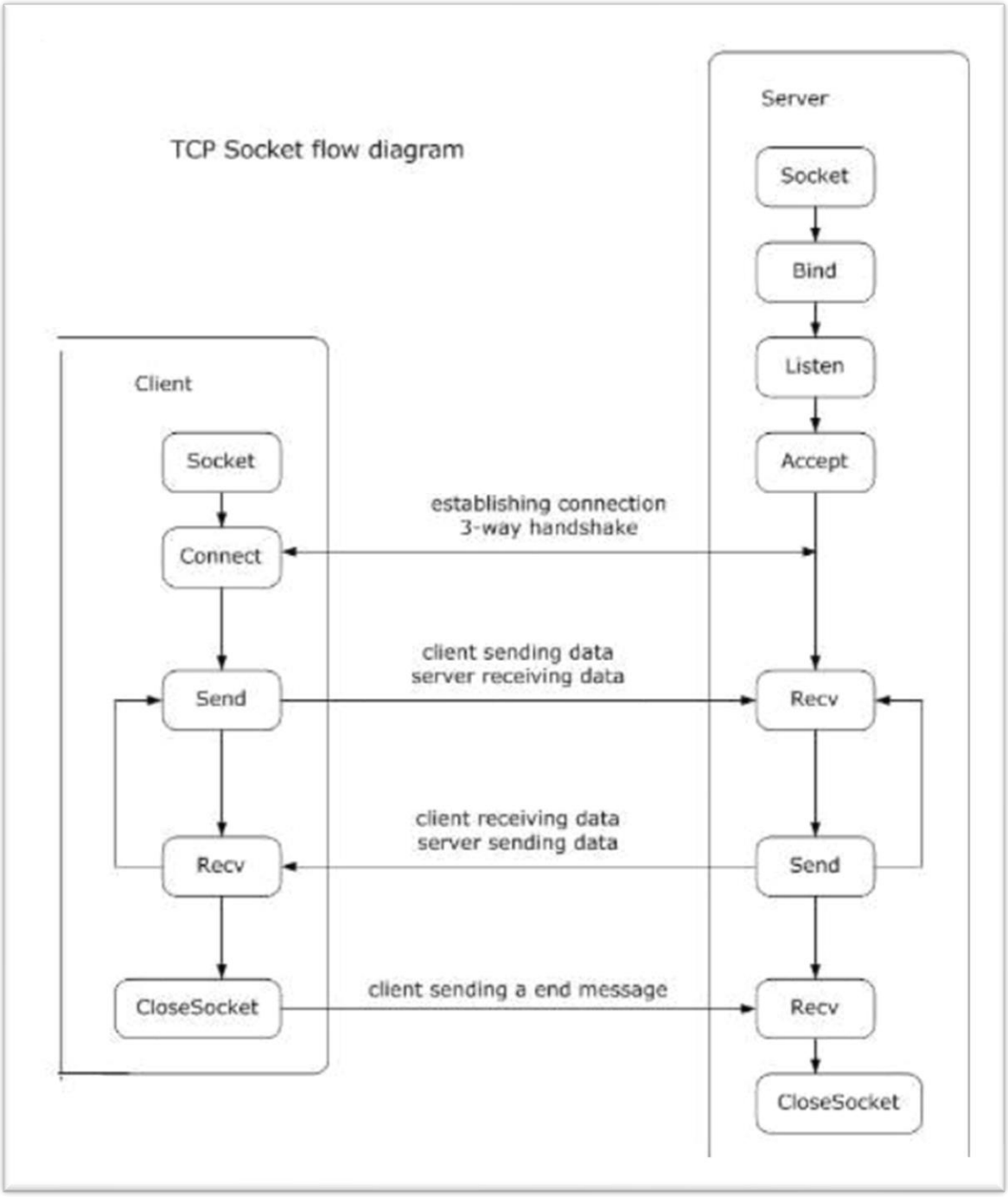
SOCK_RAW (Raw Sockets):

- **Protocol**: Used for raw network protocols.
- **Nature**: Allows sending and receiving raw packets directly over the network, bypassing standard protocols like TCP or UDP.
- **Use Case**: Developers can use raw sockets to create custom network protocols or to implement specific behaviors that are not supported by higher-level protocols like TCP or UDP.
- **Example**: Network Diagnostics and Monitoring(ping)

3.Socket Operations:

- **Socket Creation** (socket()): Creates a socket for communication.
- **Binding** (bind()): Associates a socket with a local address and port number.
- **Listening** (listen()): Prepares a socket to accept incoming connection requests (for server-side applications).

- **Accepting** (accept()): Accepts an incoming connection request from a client (for server-side applications).
- **Connecting** (connect()): Initiates a connection to a remote server (for client-side applications).
- **Sending and Receiving Data** (send(), recv(), sendto(), recvfrom()): Functions used to transmit and receive data through the socket.
- **Closing** (close()): Closes the socket and releases the resources.

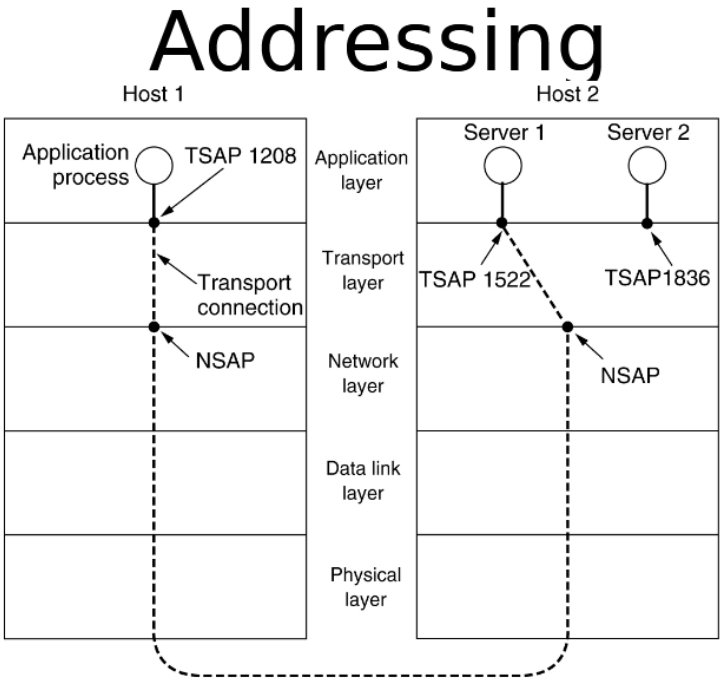


Elements of Transport Protocols:

- Addressing
- Error and Flow Control
- Multiplexing
- Crash Recovery
- Connection management

Addressing

When an application (like a user program) wants to communicate with another application over a network, it needs to know **where to send the data**. This process of specifying the destination is called **addressing**. The method normally used is to define transport addresses to which processes can listen for connection requests. In the Internet, these endpoints are called ports. We will use the generic term TSAP (Transport Service Access Point) to mean a specific endpoint in the transport layer**.TSAP (Transport Service Access Point)**:This is the **endpoint** where an application connects for sending and receiving data.A **TSAP** is like a **doorway** for applications to send or receive information. **NSAP (Network Service Access Point)**: This is the **endpoint in the network layer** that helps route data to the correct machine on the network. An **NSAP** is usually an **IP address** (like 192.168.1.10), which helps to find where data should go across the network. In the Internet, an application is identified by a **port number** (like port 80 for a website) and an **IP address** (like 192.168.1.10).



A possible scenario for a transport connection is as follows:

1. A mail server process attaches itself to TSAP 1522 on host 2 to wait for an incoming call. A call such as our LISTEN might be used, for example.

2. An application process on host 1 wants to send an email message, so it attaches itself to TSAP 1208 and issues a CONNECT request. The request specifies TSAP 1208 on host 1 as the source and TSAP 1522 on host 2 as the destination. This action ultimately results in a transport connection being established between the application process and the server.

3. The application process sends over the mail message.

4. The mail server responds to say that it will deliver the message.

5. The transport connection is released.

Error and Flow Control

Error control ensures that data is transmitted accurately and reliably, and includes:

- **Error Detection**: Typically done using a **checksum**. This allows the receiver to verify whether the data has been corrupted during transmission.
- **Error Correction**: If a segment is corrupted or lost, the receiver sends an **acknowledgment (ACK)** to the sender. If the sender doesn't receive an ACK within a specific time, it retransmits the segment.
- **Acknowledgments (ACKs)**: These messages confirm that data has been received correctly.
- **Retransmission**: If a segment is lost or corrupted, the sender will retransmit it.

Flow Control

- TCP, provides flow control. The receiver of the data controls the amount of data that are to be sent by the sender. This is done to prevent the receiver from being overwhelmed with data. This prevents congestion and ensures smooth, reliable data transfer. TCP uses a **sliding window** for flow control. This window represents the amount of data (in bytes) the sender is allowed to send before it must wait for an acknowledgment from the receiver.
- The window size adjusts dynamically based on the receiver's available buffer space, allowing more or less data to be sent at a time.
- The receiver sends the window size information back to the sender in each acknowledgment (ACK), indicating how much data the sender can transmit without waiting for further acknowledgment.
- As the receiver processes the incoming data, the available space in the buffer increases, allowing the sender to send more data.
- If the receiver's buffer fills up and it cannot process data fast enough, it will send a **smaller window size** to the sender, slowing down the data flow to avoid buffer overflow.

Multiplexing

Multiplexing and Demultiplexing in Transport Layer:

Multiplexing and Demultiplexing services are provided in almost every protocol architecture ever designed. UDP and TCP perform the demultiplexing and multiplexing jobs by including

two special fields in the segment headers: the source port number field and the destination port number field.

Multiplexing –

Gathering data from multiple application processes of the sender, enveloping that data with a header, and sending them as a whole to the intended receiver is called multiplexing.

Demultiplexing –

Delivering received segments at the receiver side to the correct app layer processes is called demultiplexing.

In the above figure, the Application layer has generated data, and then passed it down to the Transport layer to be segmented.

- After segmenting the data, port numbers are given to each segment to be ready for transmission.
- Then the segments are put on a wire to travel across the network to the destination. This process is called "multiplexing".
- When the transmitted segments reach the Transport layer of the destination, they are automatically sent up to their appropriate applications. This process is called "demultiplexing".

Crash Recovery

In network communication crash recovery plays an important role in the transport layer in ensuring the reliable exchange of data between endpoints. To achieve this reliability, various mechanisms come into play, such as retransmission, timeout-based strategies, selective repeat, and flow control. These mechanisms guarantee the efficient transmission of data, even in network failures.

The need for crash recovery arises when unforeseen disruptions, like hardware failures, network congestion, transmission errors, or packet loss, occur during network communication. These disruptions can lead to data loss or corruption, affecting the reliability and timeliness of communication services. The tools and techniques employed by the transport layer for crash recovery serve as a means to detect and rectify these disruptions, ensuring data is consistently delivered, even in the presence of network failures.

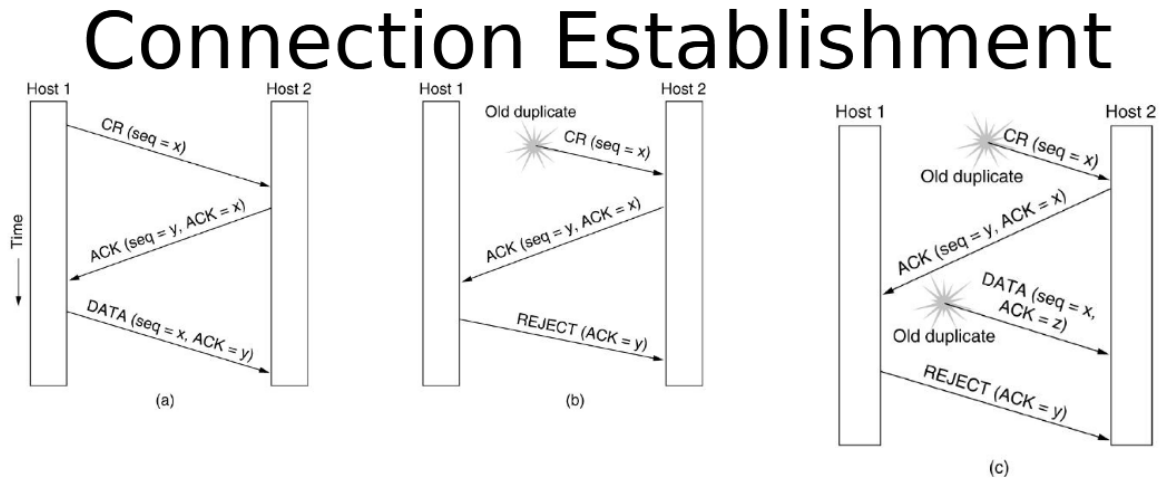
By employing techniques such as retransmission, timeout-based strategies, selective repeat, and flow control, these mechanisms maintain the integrity of data exchange between endpoints

CONNECTION MANAGEMENT: connection establishment

Establishing a connection sounds easy, but it is actually surprisingly tricky. At first glance, it would seem sufficient for one transport entity to just send a CONNECTION REQUEST segment to the destination and wait for a CONNECTION ACCEPTED reply. The problem occurs when the network can lose, delay, corrupt, and duplicate packets. This behavior causes serious complications To solve this specific problem,(DELAYED DUPLICATES) Tomlinson (1975) introduced the three-way handshake. This establishment protocol involves one peer checking with the other that the connection request is

indeed current. The normal setup procedure when host 1 initiates is shown in Fig. (a). Host 1 chooses a sequence number, x, and sends a CONNECTION REQUEST segment containing it to host 2. Host 2 replies with an ACK segment acknowledging x and announcing its own initial sequence number, y. Finally, host 1 acknowledges host 2's choice of an initial sequence number in the first data segment that it sends.

Connection Establishment



Three protocol scenarios for establishing a connection using a

three-way handshake. CR denotes CONNECTION REQUEST.

(a) Normal operation,

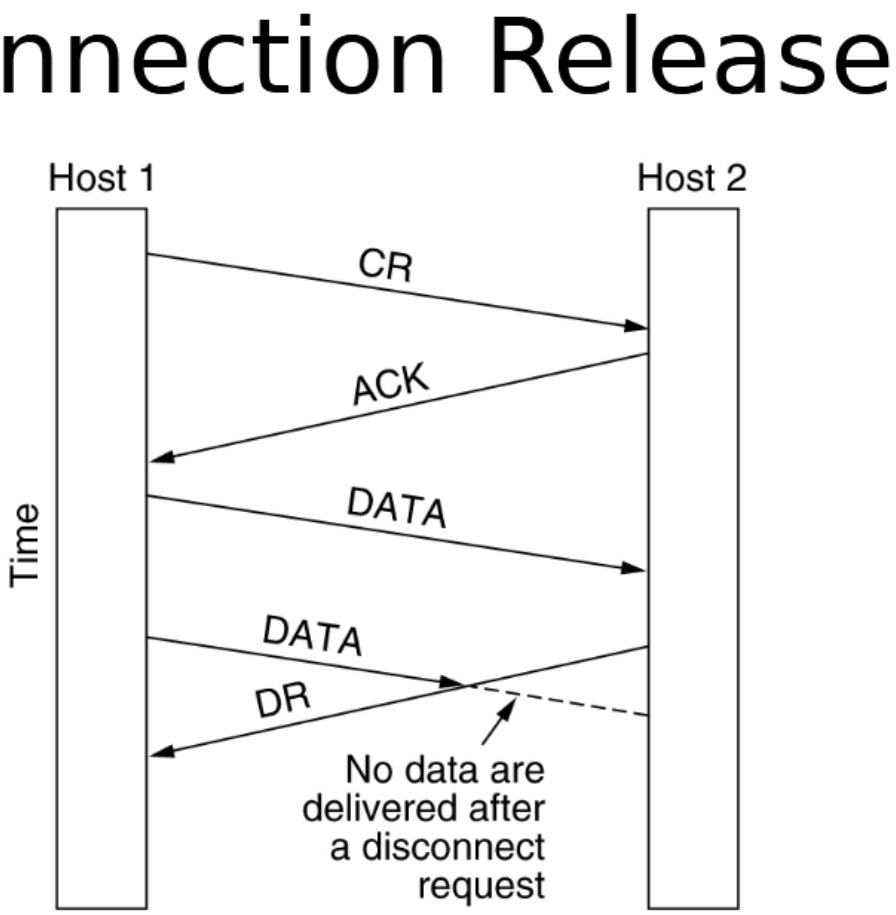
(b) Old CONNECTION REQUEST appearing out of nowhere.

(c) Duplicate CONNECTION REQUEST and duplicate ACK.

In Fig.(b), the first segment is a delayed duplicate CONNECTION REQUEST from an old connection. This segment arrives at host 2 without host 1's knowledge. Host 2 reacts to this segment by sending host 1 an ACK segment, in effect asking for verification that host 1 was indeed trying to set up a new connection. When host 1 rejects host 2's attempt to establish a connection, host 2 realizes that it was tricked by a delayed duplicate and abandons the connection. In this way, a delayed duplicate does no damage. The worst case is when both a delayed CONNECTION REQUEST and an ACK are floating around in the subnet. This case is shown in Fig. (c). As in the previous example, host 2 gets a delayed CONNECTION REQUEST and replies to it. At this point, it is crucial to realize that host 2 has proposed

using y as the initial sequence number for host 2 to host 1 traffic, knowing full well that no segments containing sequence number y or acknowledgements to y are still in existence. When the second delayed segment arrives at host 2, the fact that z has been acknowledged rather than y tells host 2 that this, too, is an old duplicate. The important thing to realize here is that there is no combination of old segments that can cause the protocol to fail and have a connection set up by accident when no one wants it.

Connection Release



Abrupt disconnection with loss of data.

There are two styles of terminating a connection: asymmetric release and symmetric release Asymmetric release is the way the telephone system works: when one party hangs up, the connection is broken. Symmetric release treats the connection as two separate unidirectional connections and requires each one to be released separately

Asymmetric release is abrupt and may result in data loss. Consider the scenario of Fig. After the connection is established, host 1 sends a segment that arrives properly at host 2. Then host 1 sends another segment. Unfortunately, host 2 issues a DISCONNECT before the second segment arrives. The result is that the connection is released and data are lost. Clearly, a more sophisticated release protocol is needed to avoid data loss. One way is to use symmetric release, in which each direction is released independently of the other one. Here, a host can continue to receive data even after it has sent a DISCONNECT segment. Symmetric release does the job when each process has a fixed amount of data to send and clearly knows when it has sent it. One can envision a protocol in which host 1 says ‘‘I am done. Are you done too?’’ If host 2 responds: ‘‘I am done too. Goodbye, the connection can be safely released.’’

Internet Transport Protocols:

The **Internet transport layer** includes protocols that ensure reliable and efficient data communication between applications across a network. The two most widely used transport protocols in the Internet are **TCP (Transmission Control Protocol)** and **UDP (User Datagram Protocol)**.

TCP SERVICE MODEL:

- The TCP (Transmission Control Protocol) service model describes how TCP ensures reliable, connection-oriented communication between two devices (or application processes) over a network. It provides a set of features that guarantee data delivery and integrity, ensuring that data is transferred correctly, in sequence, and without errors.
- Key Features of the TCP Service Model

Connection-Oriented:

- Establishes a Connection: TCP requires a three-way handshake to establish a connection before data transmission begins. This ensures that both the sender and receiver are ready to communicate.
- Connection Termination: After data transfer is complete, a four-way handshake is used to terminate the connection, ensuring that both sides agree to close the connection cleanly.

Reliable Data Transfer:

- Error-Free Data: TCP guarantees that data is delivered correctly and in the right order. If data is lost or corrupted during transmission, TCP will detect and correct the errors through retransmissions.
- Acknowledgments (ACKs): The receiver sends acknowledgments to the sender, confirming that the data has been successfully received. If the sender doesn't receive an acknowledgment within a certain time, it will retransmit the data.
- Sequence Numbers: Each segment of data sent by TCP is assigned a sequence number to maintain the correct order. This ensures that the receiver can reassemble the data in the proper order, even if segments arrive out of order.

Flow Control:

- Sliding Window: TCP uses a sliding window mechanism to manage the rate of data flow between sender and receiver. The receiver tells the sender how much data it can handle at once by adjusting the size of the window.
- Prevents Overloading: By controlling the amount of data sent before waiting for an acknowledgment, TCP ensures the receiver's buffer doesn't get overloaded.

Congestion Control:

- Avoids Network Congestion: TCP adjusts the rate of data transmission based on network congestion. It uses algorithms like Slow Start, Congestion Avoidance, and Fast Recovery to ensure that the network doesn't become overloaded.
- Dynamic Adjustment: As the network conditions change, TCP dynamically adjusts its sending rate to avoid packet loss and congestion.

Byte-Stream Service:

- Continuous Data Stream: TCP provides a byte-stream service, meaning that the application sends a continuous flow of data, which TCP breaks down into segments for transmission. The receiver reassembles the data in the correct order.

How TCP Works (Service Model in Action)

1.Connection Setup:

- The sender and receiver exchange a series of messages to establish a connection. This involves the three-way handshake:
- SYN: The sender sends a synchronization request to the receiver.
- SYN-ACK: The receiver acknowledges the request and sends a synchronization response.
- ACK: The sender confirms the acknowledgment, and the connection is established.

2.Data Transmission:

- Once the connection is established, data is transmitted in segments.
- Each segment contains a sequence number and checksum to ensure data integrity.
- The receiver sends ACKs to confirm the receipt of each segment. If a segment is lost or corrupted, the receiver does not send an acknowledgment, and the sender retransmits the segment.

3.Flow Control and Congestion Control:

- The receiver sends window size information to the sender to control the rate of data transmission (flow control).
- The sender adjusts its transmission rate based on network conditions (congestion control).

4.Connection Termination:

- Once the data transfer is complete, the connection is closed using a four-way handshake:
- FIN: The sender sends a request to terminate the connection.
- ACK: The receiver acknowledges the termination request.
- FIN: The receiver sends a request to terminate the connection from its side.
- ACK: The sender acknowledges the second termination request, and the connection is closed.

TCP Protocol:

TCP (Transmission Control Protocol) is one of the main protocols of the TCP/IP suite. It is a **connection-oriented protocol for communications** that helps in the exchange of messages between different devices over a network. It lies between the Application and Network Layers which are used in providing reliable delivery services. Transmission Control Protocol (TCP) ensures reliable and efficient data transmission over the internet. TCP plays a crucial role in managing the flow of data between computers, guaranteeing that information is delivered accurately and in the correct sequence.

Features of TCP/IP

- **Segment Numbering System:** TCP **keeps track of the segments being transmitted or received by assigning numbers to each and every single one of them**. A specific Byte Number is assigned to data bytes that are to be transferred while segments are assigned *sequence numbers*. Acknowledgment Numbers are assigned to received segments.

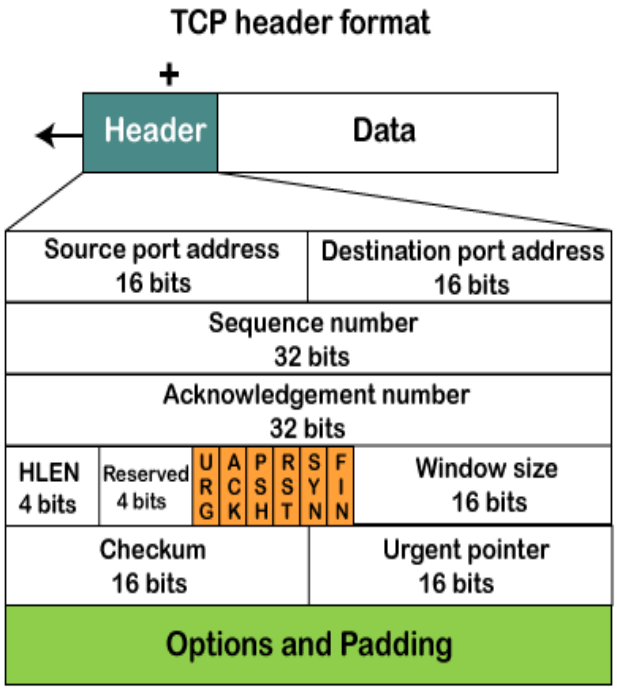
- **Connection Oriented:** It **means sender and receiver are connected to each other till the completion of the process**. The order of the data is maintained i.e. order remains same before and after transmission.
- **Full Duplex:** In **TCP data can be transmitted from receiver to the sender or vice – versa at the same time**. It increases efficiency of data flow between sender and receiver.
- **Flow Control:** Flow control limits the rate at which a sender transfers data. This is done to ensure reliable delivery. The receiver continually hints to the sender on how much data can be received (using a sliding window).
- **Error Control:** TCP implements an error control mechanism for reliable data transfer. Error control is byte-oriented. Segments are checked for error detection. Error Control includes – Corrupted Segment & Lost Segment Management, Out-of-order segments, Duplicate segments, etc.
- **Congestion Control:** TCP takes into account the level of congestion in the network. Congestion level is determined by the amount of data sent by a sender.

Applications of TCP

TCP is used for applications that require reliable and ordered delivery of data:

1. **Web Browsing (HTTP/HTTPS):** Ensures complete webpage data is received.
2. **Email (SMTP, IMAP, POP3):** Guarantees delivery of messages and attachments.
3. **File Transfers (FTP):** Maintains data integrity during transfers.
4. **Remote Access (secure shell-SSH):** Ensures secure and reliable communication.

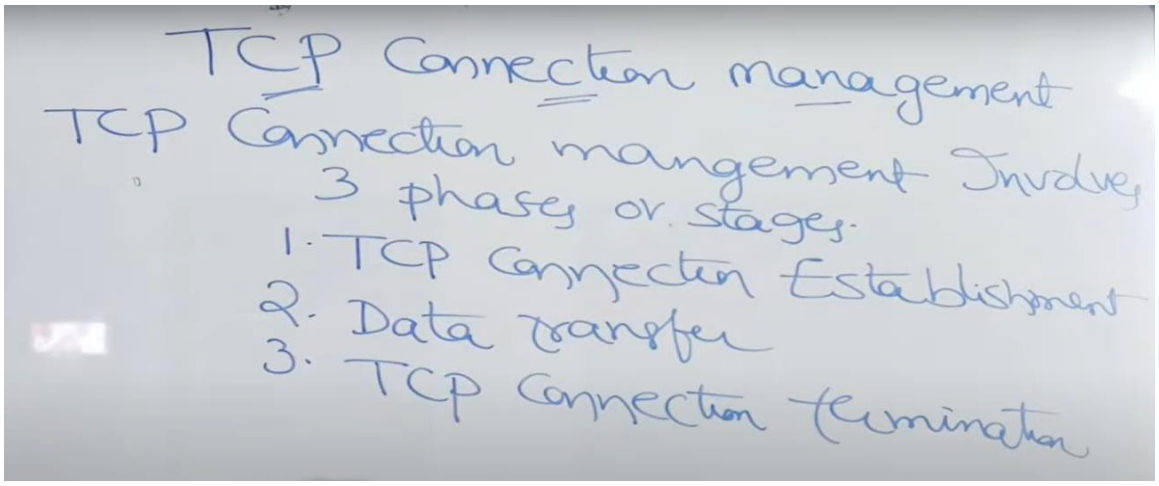
Segment Header :



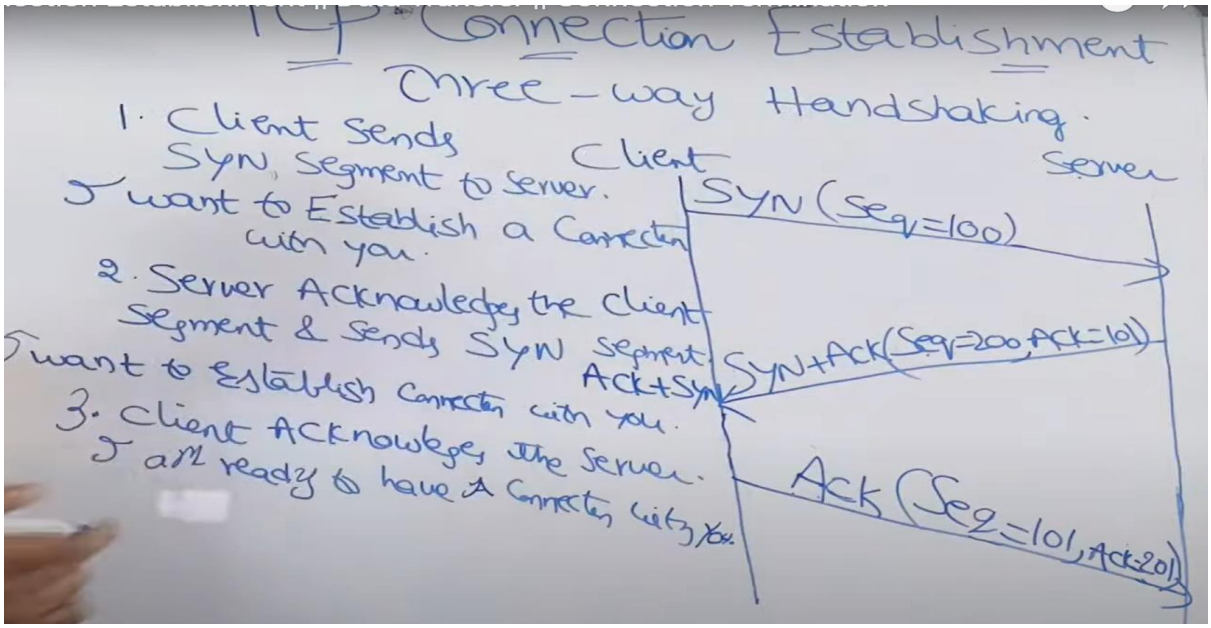
- Source port: It defines the port of the application, which is sending the data. So, this field contains the source port address, which is 16 bits.
- Destination port: It defines the port of the application on the receiving side. So, this field contains the destination port address, which is 16 bits.
- Sequence number: This field contains the sequence number of data bytes in a particular session.
- Acknowledgment number: When the ACK flag is set, then this contains the next sequence number of the data byte and works as an acknowledgment for the previous data received. For example, if the receiver receives the segment number 'x', then it responds 'x+1' as an acknowledgment number.
- HLEN: It specifies the length of the header indicated by the 4-byte words in the header. The size of the header lies between 20 and 60 bytes. Therefore, the value of this field would lie between 5 and 15.
- Reserved: It is a 4-bit field reserved for future use, and by default, all are set to zero.
- Flags
  There are six control bits or flags:
  1. URG: It represents an urgent pointer. If it is set, then the data is processed urgently.
  2. ACK: If the ACK is set to 0, then it means that the data packet does not contain an acknowledgment.
  3. PSH: If this field is set, then it requests the receiving device to push the data to the receiving application without buffering it.
  4. RST: If it is set, then it requests to restart a connection.
  5. SYN: It is used to establish a connection between the hosts.
  6. FIN: It is used to release a connection, and no further data exchange will happen.
- Window size
  It is a 16-bit field. It contains the size of data that the receiver can accept. This field is used for the flow control between the sender and receiver and also determines the amount of buffer allocated by the receiver for a segment. The value of this field is determined by the receiver.
- Checksum
  It is a 16-bit field. This field is optional in UDP, but in the case of TCP/IP, this field is mandatory.
- Urgent pointer
  It is a pointer that points to the urgent data byte if the URG flag is set to 1. It defines a value that will be added to the sequence number to get the sequence number of the last urgent byte.
- Options
  It provides additional options. The optional field is represented in 32-bits. If this field contains the data less than 32-bit, then padding is required to obtain the remaining bits.

TCP Connection establishment and Release: REFER:

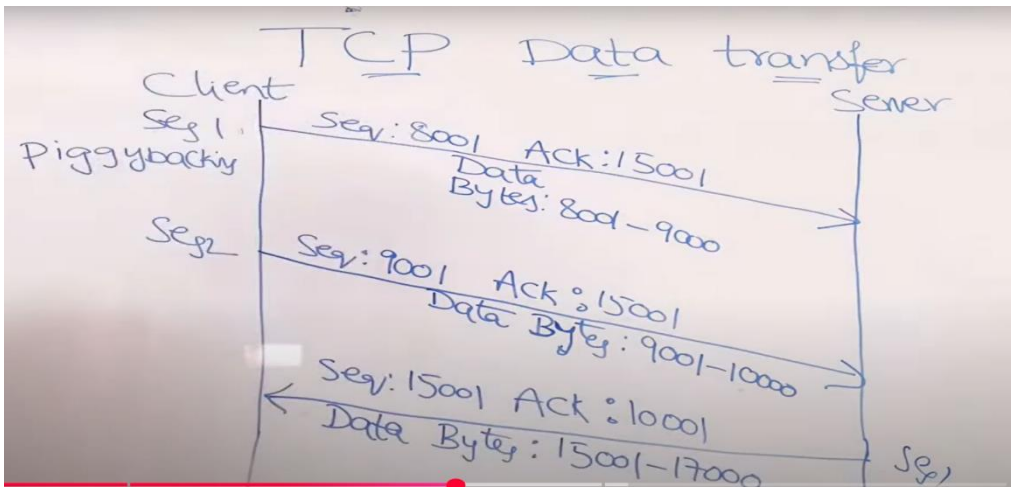
https://youtu.be/o6bQQk2pgjo?si=60BZwZ_KX2hB9FaY



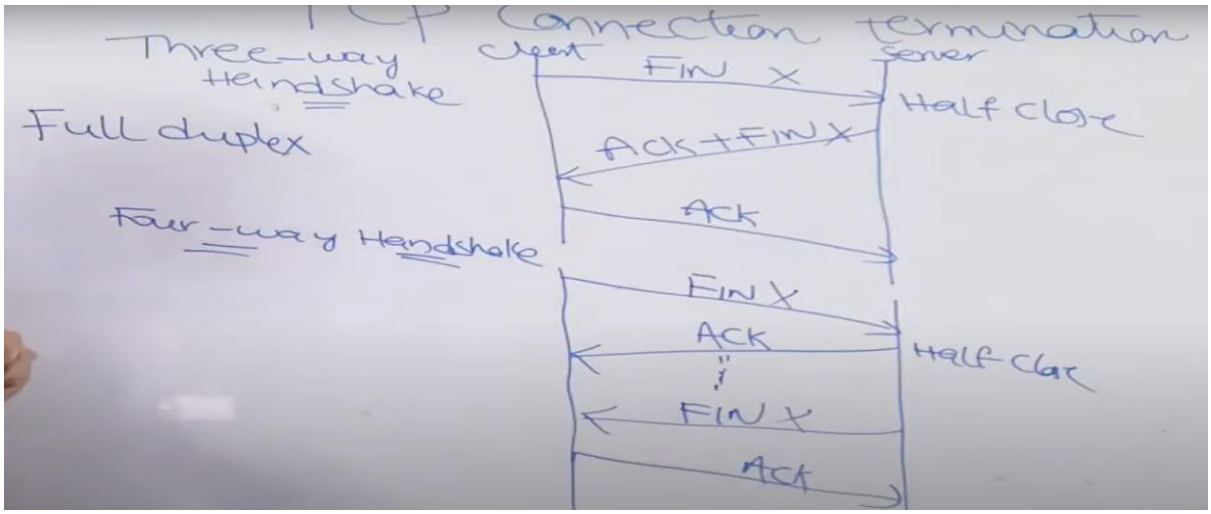
1.TCP CONNECTION ESTABLISHMENT



2.DATA TRANSFER



3.TCP CONNECTION TERMINATION



TCP uses a **three-way handshake** to establish a connection between the sender and receiver.

1. **SYN (Synchronize):**
    - The sender sends a **SYN** packet to the receiver.
    - This packet includes an **initial sequence number (ISN)**, signaling the start of communication.

Example: Sender → Receiver: "Can we connect? My ISN is 100."

2. **SYN-ACK (Synchronize-Acknowledge):**
    - The receiver acknowledges the sender's SYN by sending a **SYN-ACK** packet.
    - The receiver also sends its own ISN.

Example: Receiver → Sender: "Sure! My ISN is 200. Acknowledging your ISN 100."

3. **ACK (Acknowledge):**
    - The sender sends an **ACK** packet to confirm the connection.
    - Data transfer can now begin.

Example: Sender → Receiver: "Got it! Acknowledging your ISN 200. Let's start!"

FIN (Finish):

- The **client** sends a **FIN** (finish) message to the **server**, indicating that the client has no more data to send and wants to close the connection.This message contains a **sequence number** and indicates the client is finished sending data.

ACK (Acknowledge):

- The **server** sends an **ACK** message back to the **client**, acknowledging the receipt of the FIN.The sequence number in the ACK is **client's sequence number + 1**, confirming the receipt of the FIN.

TCP Connection Management

TCP Connection Management refers to the processes involved in establishing, maintaining, and terminating a reliable connection between two devices (hosts) over a network. TCP (Transmission Control Protocol) is connection-oriented, meaning that a connection must be established before any data can be transferred. The main goals of TCP connection management are to ensure data is transmitted reliably and in the correct order, and to properly handle the initiation and closure of the connection.

TCP connection management involves three main phases:

1. **Connection Establishment**
2. **Data Transfer**
3. **Connection Termination**

TCP Sliding Window:

In **TCP (Transmission Control Protocol)**, the **sliding window** mechanism is used to manage the flow of data between the sender and the receiver. It helps to ensure that data is sent efficiently and reliably, while also preventing the receiver from being overwhelmed by too much data at once.

How TCP Sliding Window Works

1. **Window Concept**:
    - A **window** refers to a range of sequence numbers that the sender can send without receiving an acknowledgment (ACK) from the receiver.
    - The window controls how much data can be sent at a time and prevents network congestion.
2. **Window Size**:
    - The **size of the window** is the maximum amount of data (measured in bytes or segments) that the sender is allowed to send before waiting for an acknowledgment from the receiver.
    - The window size is dynamically adjusted based on factors like available buffer space at the receiver and network conditions.
3. **Sender and Receiver Roles**:
    - The **sender** transmits data, but it can only send data up to the size of the window.
    - The **receiver** sends acknowledgments (ACKs) for the received data and indicates how much buffer space is available for new data. The receiver advertises the available space in the window, known as the **advertised window**.
4. **Sliding the Window**:
    - As the receiver acknowledges received data, the sender's window "slides" forward, allowing the next set of data to be sent.

- For example, if the sender sends a segment and gets an acknowledgment, it can slide the window forward and send the next segment in line.

Steps of the TCP Sliding Window Process

1. **Sender Sends Data**:
   - The sender sends multiple segments of data up to the size of the window.
   - These segments are numbered with sequence numbers, which help in reordering the segments and ensuring they are received correctly.
2. **Receiver Sends ACKs**:
   - The receiver receives the data and sends an acknowledgment (ACK) for the last byte of data received correctly.
   - The ACK also tells the sender the next expected byte, indicating that the receiver is ready for more data.
3. **Window Slides**:
   - Once the sender receives an acknowledgment, the window slides forward, allowing it to send more data.
   - The sliding window mechanism allows the sender to keep sending data without waiting for an acknowledgment for each individual segment, thus improving efficiency.
4. **Flow Control**:
   - The receiver's advertised window size determines how much data the sender can send at once. If the receiver has limited buffer space, it will advertise a smaller window size, slowing down the sender's transmission rate.
5. **Congestion Control**:
   - The sender also uses **congestion control** mechanisms to adjust the window size based on network congestion. If the network is congested, the sender reduces the rate at which it sends data.

TCP Timer Management

TCP uses several timers to manage different phases of the connection, such as retransmission, connection establishment, and connection termination. Timers help manage the timing of events, such as waiting for acknowledgments and handling retransmissions in case of packet loss or network delays.

Types of Timers in TCP

1. **Retransmission Timer**
   - **Purpose**: This timer is used to retransmit segments that have not been acknowledged within a certain time period.

- **How it works**: When the sender sends a TCP segment, it starts a timer. If the acknowledgment for that segment is not received before the timer expires, the segment is retransmitted.

2. **Persistent Timer**
    - **Purpose**: The **Persistent Timer** is used in TCP to handle situations where the receiver's **advertised window size** is zero. This means the receiver is temporarily unable to accept any more data because its buffer is full. Without the persistent timer, the sender would be stuck waiting indefinitely for the receiver to free up buffer space. The persistent timer prevents this by periodically probing the receiver for available space..
3. **Keep-Alive Timer**
    - **Purpose**: This timer ensures that an idle connection remains active by periodically sending small "keep-alive" probes to the other end, checking if the connection is still alive.
    - **How it works**: If the connection remains idle for a long period and no data is exchanged, the sender sends a "keep-alive" packet. If no response is received after a certain period, the connection is assumed to be lost, and the connection may be closed.
4. **Time-Wait Timer**
    - **Purpose**: The time-wait timer is used during the **connection termination** process to ensure that all delayed packets in the network are properly cleared before closing the connection.
    - **How it works**: When one side of the connection sends a FIN (finish) packet to terminate the connection, it enters the **TIME_WAIT** state. During this state, the connection remains open for a period of time (typically 2 * MSL — Maximum Segment Lifetime) to ensure that any delayed packets that might still be in the network are discarded.

TCP Congestion Control:

TCP Congestion Control is a mechanism that ensures efficient data transmission in a network by preventing congestion (overload) and ensuring that data is sent at a rate that the network can handle. Congestion occurs when there is too much data being transmitted in the network, leading to packet loss, delays, and reduced network performance. TCP uses congestion control techniques to adjust the data transmission rate and avoid overwhelming the network.

TCP Congestion Control Mechanisms

TCP uses a combination of four key algorithms to manage congestion control:

1. **Slow Start**
2. **Congestion Avoidance**
3. **Fast Retransmit**
4. **Fast Recovery**

1. Slow Start

- **Purpose**: Prevents overwhelming the network when a connection starts.
- **How it works**:
    - When a TCP connection begins, the sender starts with a small congestion window (CWND), usually 1 or 2 maximum segment sizes (MSS).
    - Each time an acknowledgment (ACK) is received for a segment, the window size increases exponentially, doubling the window size after each round-trip time (RTT) until it reaches a threshold known as the **slow start threshold** (ssthresh).
    - This exponential growth allows the sender to quickly probe for available bandwidth in the network without causing congestion.
- **Slow Start Process**:

1. Initially, CWND = 1 MSS.
2. After each ACK, the CWND is doubled (exponential growth).
3. The process continues until the CWND reaches **ssthresh**.

---

2. Congestion Avoidance

- **Purpose**: Prevents congestion after the network has ramped up in the slow start phase.
- **How it works**:
    - Once the CWND exceeds the **ssthresh**, TCP enters **Congestion Avoidance** mode.
    - In this mode, the window increases more slowly, typically linearly (i.e., it increases by 1 MSS for each RTT).
    - This helps to avoid causing network congestion while still allowing the sender to increase the transmission rate.
    - The goal is to find the optimal transmission rate that the network can handle without causing packet loss or excessive delays.
- **Congestion Avoidance Process**:

1. The CWND grows linearly, increasing by 1 MSS per RTT.
2. If the network is not congested, the sender can continue increasing the CWND slowly.

---

3. Fast Retransmit

- **Purpose**: To recover from packet loss quickly and avoid waiting for a timeout.
- **How it works**:
    - When the sender detects that a packet has been lost, it typically waits for a **timeout** to retransmit the packet. However, this can lead to unnecessary delays.

- In **Fast Retransmit**, if the sender receives **3 duplicate ACKs** (i.e., the same ACK for the same sequence number), it assumes that a packet has been lost and retransmits the lost packet immediately without waiting for the timeout.
- This helps to recover quickly from packet loss and maintain a steady flow of data.

---

4. Fast Recovery

- **Purpose**: To recover from packet loss quickly without reducing the transmission rate too much.
- **How it works**:
  - After a **Fast Retransmit**, instead of going back to **Slow Start**, the sender enters **Fast Recovery**.
  - The CWND is halved (i.e., reduced to the **ssthresh** value), but it doesn't reset to 1 MSS as in Slow Start. This allows the sender to continue transmitting without drastically reducing the window size.
  - The sender waits for further ACKs and continues transmitting at a reduced rate while the lost packet is being retransmitted.
- **Fast Recovery Process**:

1. On receiving 3 duplicate ACKs, the sender retransmits the lost packet.
2. The CWND is reduced to half the value it was before the packet loss, and the transmission rate is adjusted to avoid further congestion.
3. Once the ACK for the retransmitted packet is received, the CWND is increased again.

TCP Congestion Control Cycle

1. **Slow Start Phase**: Start with a small window and grow exponentially until **ssthresh** is reached.
2. **Congestion Avoidance Phase**: After **ssthresh** is reached, grow the window linearly to avoid congestion.
3. **Packet Loss Detected**: If packet loss occurs (e.g., through duplicate ACKs or timeout), TCP enters **Fast Retransmit** and **Fast Recovery**.
4. **Recovery**: After packet loss, the sender recovers by reducing the window and adjusting the transmission rate, continuing the process of congestion control.

UDP: User Datagram Protocol

Protocol:

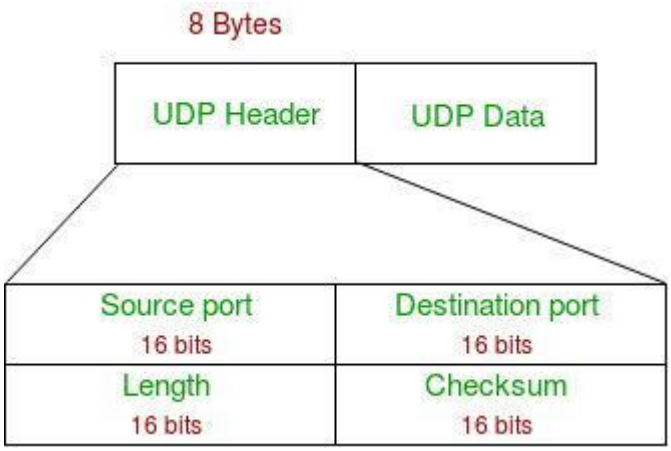
User Datagram Protocol (UDP) is a Transport Layer protocol. UDP is a part of the Internet Protocol suite, referred to as UDP/IP suite. Unlike TCP, it is an **unreliable and connectionless protocol.** So, there is no need to establish a connection before data transfer. The UDP helps to establish low-latency and loss-tolerating connections over the network. The UDP enables process-to-process communication.

Key Features of UDP

1. **Connectionless Communication:**
    - No need for connection establishment or teardown.
    - Each packet is sent independently of others.
2. **Unreliable Delivery:**
    - No retransmissions or acknowledgment of packets.
    - Suitable for scenarios where occasional data loss is acceptable.
3. **Low Overhead:**
    - Simple header structure (only 8 bytes).
    - Reduces bandwidth consumption.
4. **Fast Transmission:**
    - Ideal for applications requiring real-time data transfer, like video streaming or gaming.

UDP Header

UDP header is an **8-byte** fixed and simple header, while for TCP it may vary from 20 bytes to 60 bytes. The first 8 Bytes contain all necessary header information and the remaining part consists of data. UDP port number fields are each 16 bits long, therefore the range for port numbers is defined from 0 to 65535; port number 0 is reserved. Port numbers help to distinguish different user requests or processes.



UDP Header

- **Source Port:** Source Port is a 2 Byte long field used to identify the port number of the source.
- **Destination Port:** It is a 2 Byte long field, used to identify the port of the destined packet.
- **Length:** Length is the length of UDP including the header and the data. It is a 16-bits field.
- **Checksum:** Checksum is 2 Bytes long field. It is the 16-bit one's complement of the one's complement sum of the UDP header, the pseudo-header of information from the IP header, and the data, padded with zero octets at the end (if necessary) to make a multiple of two octets.

Differences between TCP and UDP

Basis	Transmission Control Protocol (TCP)	User Datagram Protocol (UDP)
Type of Service	TCP is a connection-oriented protocol. Connection orientation means that the communicating devices should establish a connection before transmitting data and should close the connection after transmitting the data.	UDP is the Datagram-oriented protocol. This is because there is no overhead for opening a connection, maintaining a connection, or terminating a connection. UDP is efficient for broadcast and multicast types of network transmission.
Reliability	TCP is reliable as it guarantees the delivery of data to the destination router.	The delivery of data to the destination cannot be guaranteed in UDP.
Error checking mechanism	TCP provides extensive error-checking mechanisms. It is because it provides flow control and acknowledgment of data.	UDP has only the basic error-checking mechanism using checksums.
Acknowledgment	An acknowledgment segment is present.	No acknowledgment segment.
Sequence	Sequencing of data is a feature of Transmission Control Protocol (TCP). this means that packets arrive in order at the receiver.	There is no sequencing of data in UDP. If the order is required, it has to be managed by the application layer.
Speed	TCP is comparatively slower than UDP.	UDP is faster, simpler, and more efficient than TCP.
Retransmission	Retransmission of lost packets is possible in TCP, but not in UDP.	There is no retransmission of lost packets in the User Datagram Protocol (UDP).
Header Length	TCP has a (20-60) bytes variable length header.	UDP has an 8 bytes fixed-length header.
Weight	TCP is heavy-weight.	UDP is lightweight.
Handshaking Techniques	Uses handshakes such as SYN, ACK, SYN-ACK	It's a connectionless protocol i.e. No handshake
Broadcasting	TCP doesn't support Broadcasting.	UDP supports Broadcasting.
Protocols	TCP is used by HTTP, HTTPs , FTP , SMTP and Telnet .	UDP is used by DNS , DHCP , TFTP, SNMP , RIP , and VoIP .
Stream Type	The TCP connection is a byte stream.	UDP connection is a message stream.